Plan y casos de prueba

Sistema de Reserva de Turnos | Parte 5

Asignatura: Ingeniería de Software
Integrantes: Elizeth Cedres, Nicolás Pettorosi, Lucas German, Adriano Trinidad
Docente: Andrés Gatto

1. Objetivo y alcance

Verificar validaciones, reglas de negocio y servicios; comprobar la integración con MySQL y MQTT y ejecutar los escenarios funcionales mínimos. Este documento define las condiciones y resultados esperados. Los resultados observados se presentan en Resultados e Incidencias Parte5.pdf

2. Estrategia

Nivel	Técnica	Alcance
Unitario	JUnit 5, Mockito y Bean Validation	109 ejecuciones en siete clases; sin servicios externos.
Integración y funcional	HTTP real, Docker Compose, SQL y MQTT.	Script Python reproducible y ensayos manuales de facturación y arranque.
Regresión	Mensajes inválidos seguidos de cancelación válida.	Evitar que reaparezca la desconexión del listener.

3. Entorno y criterios

Ubuntu; Java 21; Maven; Spring Boot 3.5.7; MySQL 8; Mosquitto 2. API de pruebas: http://localhost:8081/api.
Proyecto Compose: taller-parte5-test, red y volúmenes propios. Sin generador automático. Procesador cada 5 s y facturador cada 10 s solo en pruebas; configuración habitual: 60 y 300 s.
Aprobación: código HTTP esperado, estado final dentro del límite de espera y datos coherentes. Un HTTP 202 no demuestra la ejecución final. Un fallo de transporte o timeout se registra y diagnóstica; no se convierte en aprobación por reintento sin explicar la causa.
Los tests unitarios con mocks no validan SQL, bloqueos, rollback real ni entrega MQTT. La ejecución funcional no mide carga ni concurrencia entre varias instancias

4.Casos funcionales

CF-01 | Registrar establecimiento

API / MySQL; automatizada

Precondición: API disponible; nombre de prueba único.

Procedimiento: POST /establecimientos con nombre, dirección, teléfono, correo y horario 08:00-18:00.

Conservar el ID devuelto.

Resultado esperado: HTTP 201; ID positivo y datos devueltos. El establecimiento podrá asociarse a personal.

Trazabilidad: Script main(): primer POST; evidencia ejecucion-funcional.txt.

CF-02 | Registrar personal

API / MySQL; automatizada

Precondición: CF-01 completado.

Procedimiento: POST /personal con establecimiento obtenido, costo 1500, duración 30 y estado true.

Consultar PERSONAL mediante SQL.

Resultado esperado: HTTP 201. establecimiento_id del registro coincide con el establecimiento creado.

Trazabilidad: Script main(): segundo POST y consulta SQL de asociación.

CF-03 | Reservar un turno válido

API / MQTT / MySQL; automatizada

Precondición: Personal activo, horario libre y fecha futura.

Procedimiento: POST /reservas a las 10:00. Consultar GET /reservas/{id} cada 2 s, con límite de 45 s.

Resultado esperado: HTTP 202 y SOLICITADO inicial; finalmente AGENDADO.

Trazabilidad: Script reservar() y esperar(); consulta manual de persistencia para reserva 1.

CF-04 | Cancelar turno sin borrarlo

API / MQTT / MySQL; automatizada

Precondición: Reserva de CF-03 AGENDADO.

Procedimiento: DELETE /reservas/{id}. Consultar hasta CANCELADO y comprobar el registro.

Resultado esperado: HTTP 202; termina CANCELADO. El historial de la reserva se conserva.

Trazabilidad: Script y resultados-integracion.txt; reserva manual 1.

CF-05 | Confirmar disponibilidad

API; automatizada

Precondición: Personal creado; misma fecha y hora en las tres consultas.

Procedimiento: GET /personal/{id}/disponibilidad antes de reservar, después de agendar y después de cancelar.

Resultado esperado: HTTP 200: disponible true, luego false y finalmente true.

Trazabilidad: Función disponibilidad() del script; comprobaciones booleanas.

CF-06 | Evitar doble reserva

API / procesador; automatizada

Precondición: Primer turno ya AGENDADO.

Procedimiento: Solicitar un segundo turno para el mismo profesional, fecha y hora; esperar el estado definitivo antes de cancelar el primero.

Resultado esperado: 202/SOLICITADO inicial; segundo turno RECHAZADO_TURNO_OCUPADO.

Trazabilidad: Script: reserva duplicada. Ejecución: reserva 9 rechazada.

CF-07 | Profesional inexistente

API / procesador; automatizada

Precondición: Confirmar con GET que el ID 999999 devuelve 404.

Procedimiento: POST /reservas con personalId=999999, establecimiento válido y fecha futura. Consultar

estado.

Resultado esperado: Aceptación inicial 202; RECHAZADO_SOLICITUD_NO_VALIDA posterior.

Trazabilidad: Script: invalida. El identificador 999999 es una precondición, no una garantía universal.

CF-08 | Horario ocupado

API / procesador; automatizada

Precondición: Turno AGENDADO para ese horario.

Procedimiento: Repetir el horario ocupado y consultar la segunda solicitud.

Resultado esperado: RECHAZADO_TURNO_OCUPADO; primera reserva permanece agendada hasta la cancelación explícita.

Trazabilidad: Comparte ejecución con CF-06: dos requisitos cubiertos por un mismo escenario.

CF-09 | Horario fuera de agenda

API / procesador; automatizada

Precondición: Establecimiento cierra a las 18:00; duración 30 min.

Procedimiento: Solicitar a las 17:45 en fecha futura. Esperar procesamiento.

Resultado esperado: Se rechaza. En la implementación actual el estado es RECHAZADO_TURNO_OCUPADO, aunque no haya una reserva superpuesta.

Trazabilidad: Script: fuera. Registrar esta clasificación como limitación semántica.

CF-10 | Mensajes MQTT inválidos en solicitudes

MQTT / procesador; automatizada y revisión manual

Precondición: Procesador suscrito a turnos/solicitudes.

Procedimiento: Publicar esto-no-es-json, null, {}, y {"reservaId":-1}; luego crear reserva válida a las 12:00.

Resultado esperado: Mensajes descartados; reserva posterior AGENDADO. Logs contienen recepción de su ID.

Trazabilidad: Script verifica la recepción válida. El descarte individual se evidencia en los logs manuales de la sesión.

CF-11 | Reutilizar horario cancelado

API / procesador; automatizada

Precondición: CF-04 completado; disponibilidad true.

Procedimiento: Crear otra reserva al mismo horario de la cancelada.

Resultado esperado: Nueva reserva AGENDADO; la anterior continúa CANCELADO.

Trazabilidad: Script: reutilizada; resultados manuales reservas 1 y 3.

CF-12 | Recuperación en cancelaciones MQTT

MQTT / listener; automatizada y regresión unitaria

Precondición: Listener con la corrección de lectura JSON y una reserva AGENDADO.

Procedimiento: Publicar cuatro mensajes inválidos en turnos/cancelar; luego DELETE válido y consultar estado.

Resultado esperado: No se pierde conexión por esos mensajes; cancelación posterior CANCELADO y persistida.

Trazabilidad: Script, CancelarTurnoTest y evidencias antes/después de la incidencia.

5.Casos de integración

CI-01 | Primera instalación aislada

Docker / MySQL / listener; manual

Precondición: Proyecto Compose distinto y volúmenes nuevos.

Procedimiento: Arrancar primero app-listener con sus dependencias. Healthcheck ejecuta SELECT 1 por TCP.

Revisar tablas y conexión MQTT.

Resultado esperado: MySQL healthy; tablas creadas; listener activo sin Connection refused.

Trazabilidad: arranque-desde-cero.txt. Alcance: bootstrap MySQL/MQTT/listener; no representa arranque simultáneo de todos los servicios.

CI-02 | Turno atendido y facturado

Procesador / MQTT / facturador / MySQL; manual

Precondición: Reserva propia AGENDADO; costo 1500.

Procedimiento: Solo en pruebas, mover su fecha al día anterior. Esperar ciclos de 5 y 10 s; consultar estados, eventos y joins de facturación.

Resultado esperado: ATENDIDO y luego FACTURADO; cliente, factura del período del turno e ítem de 1500.

Trazabilidad: Sesión de pruebas y resultados-integracion.txt, reserva 6.

CI-03 | Evento de facturación repetido

MQTT / facturador / MySQL; manual

Precondición: CI-02 completado; un ítem y total 1500.

Procedimiento: Publicar dos veces {"reservaId":ID} en turnos/atendidos; esperar ciclo; consultar cantidad y total.

Resultado esperado: Se conserva un ítem y total 1500. No debe facturar otra vez la misma reserva.

Trazabilidad: Resultados comunicados en sesión. El ZIP conserva el estado final, no una transcripción separada del ensayo de duplicación.

CI-04 | Agrupación mensual del cliente

Facturador / MySQL; manual

Precondición: Cliente de CI-02 con factura mensual existente.

Procedimiento: Crear otro turno del mismo correo; esperar AGENDADO; mover fecha al mismo período, una hora después del primero. Esperar facturación.

Resultado esperado: Mismo cliente y factura; dos reservas FACTURADO; dos ítems de 1500; total 3000.

Trazabilidad: resultados-integracion.txt: reservas 6 y 7, factura 1

6. Inventario de pruebas unitarias

Los nombres siguientes son los métodos reales del ZIP. Algunos métodos parametrizados producen varias ejecuciones; los bucles internos de un @Test cuentan como una sola ejecución. Los totales provienen de Surefire, no del número de métodos.

EntradasValidacionTest | 14 ejecuciones

API REST; api-rest/src/test/java/uy/edu/ingsoft/api/dto/EntradasValidacionTest.java

• aceptaEstablecimientoValido

• rechazaEstablecimientoConCamposObligatoriosAusentes

• rechazaCorreoInvalidoDeEstablecimiento

• aceptaPersonalValido

• aceptaCostoCeroYPersonalInactivo

• rechazaCostoNegativo

• rechazaDuracionMenorATreinta

• rechazaDuracionMayorATreinta

• rechazaPersonalConCamposObligatoriosAusentes

• rechazaEstablecimientoNoPositivoEnPersonal

• aceptaSolicitudDeReservaValida

• rechazaReservaConCamposObligatoriosAusentes

• rechazaCorreoInvalidoEnReserva

• rechazaIdsNoPositivosEnReserva

EstablecimientoServicioTest | 14 ejecuciones

API REST;
api-rest/src/test/java/uy/edu/ingsoft/api/servicio/EstablecimientoServicioTest.java

• listaEstablecimientos

• devuelveListaVaciaCuandoNoHayEstablecimientos

• obtieneEstablecimientoExistente

• rechazaConsultaDeEstablecimientoInexistente

• creaEstablecimientoConTodosSusDatos

• noCreaConHorariosIgualesOInvertidos

• actualizaTodosLosDatosConservandoElId

• noActualizaEstablecimientoInexistente

• noActualizaConHorariosIgualesOInvertidos

• eliminaEstablecimientoSinPersonal

• impideEliminarEstablecimientoConPersonal

• noEliminaEstablecimientoInexistente

PersonalServicioTest | 14 ejecuciones

API REST; api-rest/src/test/java/uy/edu/ingsoft/api/servicio/PersonalServicioTest.java

• listaPersonalConSusDatos

• devuelveListaVaciaCuandoNoHayPersonal

• obtienePersonalExistente

• rechazaConsultaDePersonalInexistente

• listaPersonalDeEstablecimientoExistente

• rechazaListadoDeEstablecimientoInexistente

• creaPersonalAsociadoAlEstablecimiento

• noCreaPersonalSiElEstablecimientoNoExiste

• actualizaDatosYEstablecimientoConservandoElId

• noActualizaPersonalInexistente

• noActualizaSiElNuevoEstablecimientoNoExiste

• eliminaPersonalSinReservas

• impideEliminarPersonalConReservas

• noEliminaPersonalInexistente

ReservaServicioTest | 32 ejecuciones

API REST; api-rest/src/test/java/uy/edu/ingsoft/api/servicio/ReservaServicioTest.java

• listaReservas

• devuelveListaVaciaSinReservas

• obtieneReservaExistente

• rechazaConsultaDeReservaInexistente

• listaPorPersonalSinFiltroDeFecha

• filtraPorTodoElDiaSolicitado

• noListaReservasDePersonalInexistente

• registraSolicitudYPublicaSuIdentificador

• noPublicaSiFallaElGuardado

• propagaFalloDelPublicador

• publicaSolicitudDeCancelacion

• noPublicaCancelacionDeReservaInexistente

• rechazaDisponibilidadDePersonalInexistente

• personalInactivoNoEstaDisponible

• personalSinEstablecimientoNoEstaDisponible

• personalConDuracionIncompatibleNoEstaDisponible

• rechazaTurnosFueraDelHorario

• aceptaLosLimitesDelHorario

• horarioSinReservasEstaDisponible

• estadosQueOcupanElHorarioImpidenReservar

• estadosQueNoOcupanElHorarioPermitenReservar

• rechazaSolapamientoParcial

• permiteTurnosContiguosSinSolapamiento

CancelarTurnoTest | 8 ejecuciones

Proyecto principal;
src/test/java/com/mycompany/taller2/ingsoft/CancelarTurnoTest.java

• descartaMensajeInvalidoSinAccederALaBase

• procesaCancelacionValidaDespuesDeMensajeInvalido

FacturadorAutomaticoTest | 11 ejecuciones

Proyecto principal;
src/test/java/com/mycompany/taller2/ingsoft/FacturadorAutomaticoTest.java

• noFacturaReservaSinCorreo

• noFacturaReservaConCorreoEnBlanco

• noFacturaReservaSinPersonal

• agregaItemConImporteYReservaCorrectos

• acumulaElImporteSobreElTotalExistente

• noDuplicaItemSiLaReservaYaTieneUno

• reutilizaClienteYFacturaExistentes

• actualizaTelefonoDelClienteExistente

• conservaTelefonoSiLaReservaNoIndicaUno

• creaClienteYFacturaCuandoNoExisten

• buscaFacturaDelPeriodoDelTurno

ProcesadorSolicitudesTest | 16 ejecuciones

Proyecto principal;
src/test/java/com/mycompany/taller2/ingsoft/ProcesadorSolicitudesTest.java

• aceptaSolicitudConRelacionYFechaValidas

• rechazaSolicitudSinEstablecimiento

• rechazaSolicitudSinPersonal

• rechazaEstablecimientoInexistente

• rechazaPersonalInexistente

• rechazaPersonalDeOtroEstablecimiento

• rechazaFechaPasada

• rechazaFechaAusente

• horarioSinReservasEstaDisponible

• reservaEnElMismoHorarioImpideAgendar

• turnoContiguoNoSeSolapa

• rechazaTurnoQueTerminaDespuesDelCierre

• personalInactivoNoEstaDisponible

• marcaComoAtendidoUnTurnoFinalizado

• mantieneAgendadoUnTurnoFuturo

• mantieneAgendadoUnTurnoQueAunNoTermino